

Bill collection tracker — POC implementation plan

Sep 23, 2026 · @Someone

Email-only proof of concept on AWS (existing free-tier/credit account), built for ThoorigAI Infotech to hand to a developer as a self-contained build brief.

Scope

Proves the core loop end to end: a client creates a bill collection workflow, alerts move stage by stage by email, approvers act via a no-login magic link, reminders fire on schedule, and payments allocate against individual bills including partial payment.

In scope: workflow builder (sequential stages, contacts), batch and bill creation, email alerts only, magic-link approver actions (complete, return, reassign), configurable reminders and escalation, client notifications, bill-level payment allocation with deductions, timeline dashboard, audit trail.

Out of scope for this POC: WhatsApp, SMS, voice calls, AI bill extraction, accounting integrations, multi-tenancy, parallel approvals, high-availability infrastructure. These are documented in the full product plan and can be layered on once the POC validates the workflow with real approvers.

Target timeline: 3–4 weeks for one developer, working solo through the phases below in order.

Architecture

flowchart LR

```
Admin[Client admin] --> Web[Next.js web<br/>on EC2]
Approver[Approver<br/>opens email link] --> Web
Web --> API[NestJS API<br/>on EC2]
API --> DB[(RDS PostgreSQL)]
API --> S3[(S3 bucket<br/>bill files, proofs)]
API --> Cron[node-cron sweeper<br/>in-process]
Cron --> API
API --> Gmail[Gmail SMTP<br/>via Nodemailer]
Gmail --> Inbox[Approver's<br/>email inbox]
```

Everything except Gmail runs inside your AWS account in `ap-south-1` : one EC2 instance hosts the web app, the API and the reminder sweeper together; RDS holds all data; S3 holds bill files. Gmail is the only external dependency, standing in for AWS SES in this version of the plan. This is the same shape as the earlier AWS option, simplified to one instance and one email provider for a fast, low-cost POC.

Prerequisites

Get these ready before the developer starts, so no phase blocks on waiting.

1. **AWS account access** — IAM user or role with permissions for EC2, RDS, S3, SES, VPC and IAM. Confirm current plan and credit balance in Billing.
2. **A Gmail account for sending** — use a dedicated Gmail address (not a personal inbox), e.g. `thoorigai.bills@gmail.com`. Turn on 2-Step Verification, then generate an **App Password** (Google Account → Security → 2-Step Verification → App passwords) for the app to send through. No domain purchase needed for the POC.
3. **Know Gmail's limits going in** — 500 emails/day on a free Gmail account, no bounce-tracking webhook. Fine for POC volume; flagged again in Phase 5 and Phase 11 so it isn't a surprise later.
4. **GitHub (or GitLab) organization** for the repo, with the developer added as a collaborator.
5. **A test workflow from the client** — one real customer org's approval chain (names, roles, email addresses, typical bill volume) to seed realistic test data. This is the single most valuable input the client can provide before build starts.
6. **Budget alerts** set in AWS Budgets at two thresholds (e.g. ₹5,000 and ₹15,000) so spend is visible early.

Phase 1 — AWS foundation (days 1-2)

1. Create a VPC (or use the account default) in `ap-south-1` with one public subnet. No NAT gateway — keep the app instance in the public subnet with a tight security group.
2. Launch one **EC2 t4g.micro or t4g.small** (Amazon Linux 2023 or Ubuntu 24.04, ARM/Graviton). Security group: inbound 22 (SSH, your IP only), 80/443 (public), all outbound.
3. Allocate an Elastic IP and attach it, so the address survives instance stop/start.
4. Install Docker and Docker Compose on the instance.

5. Create an **RDS PostgreSQL db.t4g.micro**, single-AZ, in the same VPC. Enable automated backups with a 7-day retention window. Security group: inbound 5432 from the EC2 instance's security group only, never from the public internet.
6. Create an **S3 bucket** for bill files and payment proofs, private by default, with server-side encryption (SSE-S3) enabled.
7. Create the **Gmail App Password** (see Prerequisites) and store it securely — you'll add it as an environment variable in Phase 2, never commit it to the repo.
8. Create an **IAM role** for the EC2 instance with least-privilege access to the S3 bucket only. Attach it to the instance — never put long-lived AWS keys in application config.
9. No domain is required for this POC. If you want a real URL instead of the raw IP for the approver links (recommended — looks more trustworthy in an email), use a free option like DuckDNS or nip.io pointed at the Elastic IP, or buy a cheap domain later (~₹1,000/year) with no rework needed.
10. Set up **AWS Budgets** alerts as described in Prerequisites.

Phase 2 — repo, tooling, environments (days 2-3)

1. Create a monorepo with three top-level folders: `api` (NestJS), `web` (Next.js), `infra` (Docker Compose files, deployment scripts).
2. Scaffold NestJS with modules for `workflows`, `batches`, `bills`, `payments`, `notifications`, `reminders`, `audit` and `auth`. Keep each module's controller, service and DTOs together.
3. Scaffold Next.js for two areas: an authenticated admin app (client dashboard) and an unauthenticated approver page reached only via signed magic links.
4. Set up **TypeORM** or **Prisma** against PostgreSQL, with migrations checked into the repo (never rely on auto-sync in anything beyond local dev).
5. Create three environment configs: `local` (Docker Compose with Postgres + app, no AWS dependency), `poc` (points at the real RDS and SES), and `.env.example` documenting every required variable.
6. Add `docker-compose.yml` for local development: Postgres, the API, and the web app, so the developer never needs AWS access to write and test code day to day.
7. Set up ESLint, Prettier and a pre-commit hook (Husky) so code style is consistent from the first commit.
8. Add a `README.md` documenting local setup in under five commands.

Phase 3 — data model and migrations (days 3–5)

Core tables, in build order:

Table	Key columns	Notes
<code>organizations</code>	<code>id</code> , <code>name</code>	The client's customer org this workflow belongs to
<code>contacts</code>	<code>id</code> , <code>org_id</code> , <code>name</code> , <code>email</code> , <code>role</code>	People who can be assigned to a stage
<code>workflow_templates</code>	<code>id</code> , <code>org_id</code> , <code>name</code>	One per customer org
<code>workflow_stages</code>	<code>id</code> , <code>template_id</code> , <code>order</code> , <code>contact_id</code> , <code>reminder_interval_hours</code> , <code>max_reminders</code> , <code>backup_contact_id</code>	Ordered stages with per-stage reminder config
<code>batches</code>	<code>id</code> , <code>org_id</code> , <code>template_id</code> , <code>status</code> , <code>created_at</code>	A submission of one or more bills
<code>bills</code>	<code>id</code> , <code>batch_id</code> , <code>bill_number</code> , <code>amount</code> , <code>status</code> , <code>balance_due</code>	status: pending / partially_paid / paid / disputed
<code>step_instances</code>	<code>id</code> , <code>batch_id</code> , <code>stage_id</code> , <code>status</code> , <code>alerted_at</code> , <code>next_reminder_at</code> , <code>reminder_count</code> , <code>completed_at</code>	status: pending / alerted / completed / returned / escalated
<code>payments</code>	<code>id</code> , <code>batch_id</code> , <code>amount</code> , <code>paid_at</code> , <code>reference</code> , <code>proof_url</code>	One payment can cover several bills
<code>payment_allocations</code>	<code>id</code> , <code>payment_id</code> , <code>bill_id</code> , <code>amount</code> , <code>deduction_type</code> , <code>deduction_amount</code>	Links a payment to specific bills; <code>deduction_type</code> covers TDS, retention, penalty
<code>audit_log</code>	<code>id</code> , <code>entity_type</code> , <code>entity_id</code> , <code>action</code> , <code>actor</code> , <code>detail</code> , <code>created_at</code>	Append-only, every state change writes here

Write the first migration to create all tables with foreign keys and indexes on `batch_id`, `next_reminder_at`, and `org_id`. Seed a script that loads the client's real test workflow from Prerequisites step 5, so early testing uses realistic data rather than placeholder names.

Phase 4 — backend core: auth, workflow engine (days 5–9)

1. **Client auth** — simple email + password or magic-link login for the client's own dashboard (this is separate from the approver magic links). JWT session, bcrypt-hashed passwords if using password auth.
2. **Workflow CRUD** — endpoints to create/edit a workflow template, add/reorder/remove stages, assign contacts.
3. **Batch and bill creation** — endpoint to create a batch against a template, attach one or more bills (manual entry for the POC; CSV import is a nice-to-have, not required).
4. **State machine** — implement step transitions as an explicit service, not scattered conditionals: `pending` → `alerted` → `completed`, with `returned` and `escalated` as branches. Every transition writes one row to `audit_log` with actor, timestamp and detail. This is the design's backbone — get it right before building UI on top of it.
5. **Batch creation triggers stage 1** — on batch creation, create `step_instances` for every stage (all `pending`), set stage 1 to `alerted`, set its `next_reminder_at`, and fire the first email (Phase 5).
6. **Step completion advances the batch** — completing step N sets step N+1 to `alerted`, fires its email, and notifies the client (reuse the same email adapter with a different template). Completing the last step moves the batch to a `ready_for_payment` status.
7. Write unit tests for the state machine transitions before moving to Phase 5 — this logic is the hardest to fix later once UI and emails depend on it.

Phase 5 — email integration via Gmail (days 9–11)

1. Build a thin **notification adapter interface** (`send(to, template, data)`) even though only Gmail/email is wired for the POC — this is what makes adding WhatsApp, SMS or a swap to SES later a new adapter, not a rewrite.
2. Integrate **Nodemailer** with Gmail's SMTP settings: host `smtp.gmail.com`, port `587`, `secure: false` (STARTTLS), auth using the Gmail address and the App Password from Prerequisites — never the account's real login password.
3. Store the Gmail address and App Password as environment variables (`EMAIL_USER`, `EMAIL_APP_PASSWORD`), loaded via the Parameter Store setup in Phase 10, not hardcoded.
4. Build three email templates to start: stage alert ("a bill is waiting on you"), reminder (same, marked as a follow-up), and client notification ("step completed, moved to X"). Plain HTML with inline CSS — keep it simple for the POC.

5. Every outbound email logs a row (`notifications` table: `batch_id`, `step_id`, `channel`, `recipient`, `sent_at`, `status`) so delivery is traceable from the dashboard, since Gmail gives no bounce webhook to rely on instead.
6. **Known limitation to flag to the client:** Gmail's free tier caps at 500 sends/day (fine for POC volume) and gives no bounce/complaint signal, so a wrong email address fails silently. This is the trade-off for skipping domain setup and SES — documented again in Phase 11 and the handover checklist as a fast-follow once the POC is validated.
7. Test end-to-end: create a batch, confirm stage 1's contact receives a real email with a working link, and check it doesn't land in spam.
1. Build a thin **notification adapter interface** (`send(to, template, data)`) even though only SES/email is wired for the POC — this is what makes adding WhatsApp or SMS later a new adapter, not a rewrite.
2. Integrate **AWS SES** using the AWS SDK, sending from the verified domain address.
3. Build three email templates to start: stage alert ("a bill is waiting on you"), reminder (same, marked as a follow-up), and client notification ("step completed, moved to X"). Plain HTML with inline CSS — keep it simple for the POC.
4. Every outbound email logs a row (`notifications` table: `batch_id`, `step_id`, `channel`, `recipient`, `sent_at`, `status`) so delivery is traceable from the dashboard.
5. Handle SES bounce and complaint notifications via an SNS topic subscribed to an API endpoint, so a bad email address is flagged rather than silently retried forever.
6. Test end-to-end: create a batch, confirm stage 1's contact receives a real email with a working link.

Phase 6 — approver magic-link flow (days 11-14)

1. Generate a **signed, expiring token** (JWT or HMAC-signed opaque token) per step instance when it's alerted — valid until the step is completed, not on a short fixed expiry, since email security scanners can pre-click links and consume single-use ones.
2. Build the approver page (`/approve/:token`) as an unauthenticated Next.js route: shows the batch's bills, amounts, and any notes from prior stages.
3. Add an **OTP step** for extra assurance: on page load, send a 6-digit code to the contact's email; require it before showing the action buttons. This guards against a forwarded link being actioned by the wrong person.

4. Three actions: **Complete** (advances the workflow), **Return with comment** (sends the batch back to the previous stage with a required note), **Reassign** (client-approved only for POC — flag it rather than auto-executing).
5. Every action writes to `audit_log` with the token's contact, IP address, and timestamp.
6. Add a **"confirm on behalf of"** action in the admin dashboard, so the client can mark a step complete when confirmation came by phone or offline — tag these entries with `source: manual` in the audit log so they're visibly different from a real click.

Phase 7 — reminder and escalation engine (days 14–16)

1. Add `next_reminder_at` to `step_instances` (already in the schema) and a **sweeper job**: a scheduled function that queries steps where `status = 'alerted' AND next_reminder_at <= now()`.
2. Since the EC2 instance runs continuously for this POC, use **node-cron** inside the API process to run the sweeper every 1–2 minutes — no separate scheduler service needed at this scale.
3. Sweeper logic per due step: send the reminder email, increment `reminder_count`, and either set the next `next_reminder_at` (per-stage `reminder_interval_hours`) or, if `reminder_count >= max_reminders`, mark the step `escalated` and notify the backup contact and the client.
4. Make the sweeper **idempotent and safe under concurrency** — use a `SELECT ... FOR UPDATE SKIP LOCKED` query or an equivalent row lock, so a slow run never double-sends if triggered twice.
5. Add configurable **quiet hours** (e.g. no reminders 9pm–8am) at the org or stage level, checked before sending.
6. Log every reminder sent to the `notifications` table (Phase 5) so the dashboard timeline shows reminder history, not just the original alert.

Phase 8 — admin frontend (days 16–20, overlaps with Phase 7)

1. **Login** page for the client.
2. **Workflow builder** — create/edit a template, add stages in order, assign contacts with name/email/reminder settings.
3. **Batch creation** — pick a workflow, add bills (bill number, amount), submit.

4. **"Currently with" dashboard** — the core screen: every active batch, its current stage, who it's with, how long it's been there, and a visual indicator for overdue steps (past reminder threshold).
5. **Batch detail view** — full timeline for one batch: every stage, its status, timestamps, comments from returns, and the audit trail.
6. **Contacts management** — add/edit people and their org affiliation, reused across workflows.
7. Keep styling simple and functional for the POC — a clean table-and-card layout is enough; polish comes after the client validates the workflow itself.

Phase 9 — payments and bill-level allocation (days 20–23)

1. Once a batch reaches `ready_for_payment`, show it in a **payment review screen** listing every bill with its amount and current balance.
2. **Record a payment** — amount received, date, reference/UTR, optional proof upload (to S3).
3. **Allocation UI** — for each bill in the batch, an editable "amount applying now" field. Validate that the sum of allocations plus any deductions equals the payment total before saving.
4. **Deductions** — per allocation, an optional deduction type (TDS, retention, penalty, other) and amount, so the bill's recorded balance reflects reality even when the transferred amount is lower than the invoice.
5. **Unallocated state** — if a payment arrives with no bill specified, record it as `on_account` against the batch and let the client allocate it later.
6. Update each bill's `status` and `balance_due` automatically as allocations are saved: `paid` when balance reaches zero, `partially_paid` otherwise.
7. Show a clear partial-payment summary on the batch detail view: which bills are fully paid, which are partial (with balance), and which are untouched.

Phase 10 — deployment and CI/CD (days 23–25, can start earlier in parallel)

1. Write a production `docker-compose.yml` on the EC2 instance: API container, web container (or serve both behind one Nginx reverse proxy container), no Redis needed since reminders run in-process.

2. Add **Nginx** as a reverse proxy with a free **Let's Encrypt (Certbot)** TLS certificate for the domain.
3. Set up a simple **GitHub Actions** workflow: on push to `main`, run tests, build Docker images, SSH into the EC2 instance, pull and restart containers. No blue-green deployment needed for a POC — brief downtime on deploy is acceptable.
4. Store secrets (DB credentials, SES config, JWT secret) in **AWS Systems Manager Parameter Store** (free tier) rather than in the repo or plain `.env` files on the server.
5. Add a **health check endpoint** (`/health`) that the deploy script polls after restart, to catch a failed deploy immediately.
6. Document the manual deploy and rollback steps in the README, in case CI/CD isn't finished before the first demo is needed.

Phase 11 — QA and test plan (days 25–27)

1. **Unit tests** for the workflow state machine (Phase 4) and the payment allocation math (Phase 9) — these two areas carry the most business logic and the most risk if wrong.
2. **Integration test**: create a batch → confirm email received → complete via magic link → confirm stage 2 alerted and client notified → repeat to the last stage → record a partial payment → confirm bill balances update correctly.
3. **Reminder test**: create a batch, manually backdate a step's `next_reminder_at`, confirm the sweeper sends a reminder and increments the count; force `max_reminders` and confirm escalation fires.
4. **Load sanity check**: create 20–30 batches with multiple bills each, confirm the dashboard and sweeper remain responsive — this is a POC, not a load test, but it should not visibly struggle with realistic pilot volume.
5. **Manual test with the client's real workflow** (from Prerequisites step 5) using real or test email addresses for each stage contact, before the demo.
6. Fix any deliverability issues (spam folder landing, broken links, OTP failures) found during the manual test — these are the failures most likely to undermine trust in the demo. Gmail-sent mail is more likely to land in spam than a domain-verified sender, so check every test inbox's spam folder, not just the inbox.

Phase 12 — demo script and handover checklist (day 27–28)

Demo script (walk the client through this live):

1. Show the workflow builder with their real customer org's approval chain already configured.
2. Create a new batch with two or three sample bills, submit it.
3. Open the stage 1 contact's inbox live, show the email, click through to the approver page, complete OTP, mark complete.
4. Show the client dashboard update in real time — stage moved, client notified.
5. Show a reminder firing (pre-arrange a step with a short reminder interval for the demo, or trigger it manually).
6. Walk the batch to the final stage and record a partial payment, showing the bill-level allocation and remaining balance.
7. Show the audit trail for the full batch.

Handover checklist:

- ☐ Repo access transferred/shared with the client's chosen point of contact
- ☐ README covers local setup, environment variables, and deploy steps
- ☐ AWS account costs to date and remaining credit balance shared
- ☐ Known limitations documented (Gmail sending with a 500/day cap and no bounce tracking, no HA, sandbox-level scale)
- ☐ Next-phase options discussed: WhatsApp/SMS, accounting integration, multi-tenancy — see the earlier cost and architecture options for AWS, GCP and free-tier